

Hands-On Lab (HOL)

Creating a Docker Image from a Running Container

Trainer: Dr. Sandeep Kumar Sharma

Experience: 12+ Years in Cloud, DevOps, and Data Platforms

1. Learning Objectives

By the end of this lab, participants will:

- Understand what a Docker Image is.
 - Understand the relationship between Docker images and containers.
 - Learn how to create an image from an existing container.
 - Understand when and why we create images using containers.
 - Practice using the `docker commit` command.
-

2. Learning Outcomes

After completing this lab, participants will be able to:

- Explain the purpose of Docker images.
 - Convert a running or stopped container into a Docker image.
 - Inspect Docker images.
 - Launch new containers from custom images.
-

3. Concept Explanation

In Docker, an **image** is basically a blueprint used to create containers.

Think about it like this:

- **Docker Image** → Blueprint
- **Docker Container** → Running instance created from that blueprint

Whenever we run a container, Docker first looks for the image. If the image exists locally it uses it, otherwise Docker downloads it from Docker Hub.

Normally images are created using a **Dockerfile**, but sometimes we create images from containers when we manually configure a container environment and want to save that state.

For example:

- installing software
- configuring tools
- debugging environments

Once everything is working properly, we can convert that container into an image using the `docker commit` command.

This image can then be reused to create multiple containers.

4. Lab Scenario

In this lab we will:

1. Run an Ubuntu container
 2. Install packages inside the container
 3. Save the container as a Docker image
 4. Inspect the image
 5. Create a new container from the created image
-

Step 1: Verify Docker Installation

Check whether Docker is installed on the system.

```
docker --version
```

Step 2: Check Existing Containers

List all containers including stopped containers.

```
docker ps -a
```

Step 3: Run a Base Container

Now we will run an Ubuntu container.

```
docker run -it --name basecontainer ubuntu
```

Explanation:

- `-it` → interactive terminal
- `--name` → container name
- `ubuntu` → base image

If the image is not present locally, Docker will automatically pull it.

Step 4: Modify the Container

Now we are inside the container shell. Let's make some changes to the container.

Run the following commands **inside the container**:

```
apt update  
apt install nginx -y  
apt install curl -y
```

Verify nginx installation:

```
nginx -v
```

Exit the container:

```
exit
```

Now the container has been modified with additional packages.

Step 5: Verify Container State

Check the container again.

```
docker ps -a
```

Step 6: Check Available Images

```
docker images
```

At this stage, only the **ubuntu image** should be visible.

Step 7: Create an Image from the Container

Now we will convert the container into an image.

```
docker container commit -a "sandeep" -m "This is my first docker image"
basecontainer sandeep_image
```

Explanation:

- `-a` → author name
- `-m` → commit message
- `basecontainer` → source container
- `sandeep_image` → new image name

Step 8: Verify the New Image

```
docker images
```

You should now see a new image named **sandeep_image**.

Step 9: Inspect the Image

Inspect the image to view metadata.

```
docker inspect sandeep_image
```

This command shows detailed information such as:

- image layers
 - creation time
 - configuration
 - environment details
-

Step 10: Remove the Old Container

Now remove the old container.

```
docker rm -f basecontainer
```

Verify:

```
docker ps -a
```

Step 11: Run a New Container from the Image

Now create a new container from the newly created image.

```
docker run -it --name newcontainer sandeep_image
```

Inside the container verify nginx:

```
nginx -v
```

You will notice that nginx is already installed because it was saved in the image.

5. Real World Use Case

Creating images from containers is useful in the following scenarios:

- Saving a manually configured environment
- Capturing debugging environments
- Sharing a working setup with other developers
- Rapid prototyping

However, in production environments, images are usually created using **Dockerfiles** because they are reproducible and version-controlled.

6. Lab Summary

In this lab we learned:

- What Docker images are
- The relationship between containers and images
- How to modify a running container
- How to create an image using `docker commit`
- How to run containers from a custom image

This technique is helpful for quickly capturing container environments and reusing them later.